Software Engineering Lab

Lab 26

Contract Testing

Consumer-Driven Contracts, CI Deployment Gate, dan
Safe API Evolution tanpa E2E Rapuh



Lukman / lukman-ss

Unit Test Hijau Bukan Berarti Integrasi Selamat

Provider mengubah kontrak tanpa koordinasi consumer — deploy berhasil, production gagal.

TIGA MUKA KEGAGALAN INTEGRASI

Field Rename Tersembunyi

customer.name diganti customer.full_name di provider. JSON valid, tapi consumer mendapat null.

TIGA MUKA KEGAGALAN INTEGRASI

Enum Casing Change

Status IN_PROGRESS berubah jadi in_progress. Response tetap 200 OK, tapi consumer gagal parsing.

TIGA MUKA KEGAGALAN INTEGRASI

Primitive Type Mutation

total integer 150000 jadi string '150000'. Consumer math operations pecah di runtime.

Ketika Service A dan Service B masing-masing lulus unit test, integrasi mereka tetap bisa pecah di produksi karena provider mengubah format pesan tanpa sepengetahuan consumer. Tiga skenario di atas menunjukkan bahwa JSON yang valid secara sintaksis tidak menjamin kompatibilitas semantik.

Unit test green status is not sufficient to guarantee integration safety across service boundaries.

End-to-End Test dan Manual Review Tidak Otomatis Menangkap Breaking Change

E2E lambat dan mahal; contract testing memberikan feedback cepat di setiap PR.

KELEMAHAN PENDEKATAN TRADISIONAL

E2E Test Mahal

Menjalankan seluruh sistem gabungan butuh environment lengkap, data fixture, dan waktu build lama.

KELEMAHAN PENDEKATAN TRADISIONAL

Manual Code Review

Inspeksi manual perubahan API response tidak otomatis menemukan field rename atau type change.

KELEMAHAN PENDEKATAN TRADISIONAL

Tidak Otomatis Aman

Deploy pipeline tanpa gate verifikasi kontrak mengizinkan breaking change lolos ke produksi.

End-to-end test mensimulasikan seluruh alur sistem sehingga mahal untuk dijalankan setiap PR. Review manual pun tidak otomatis menemukan perubahan breaking karena developer fokus pada logika bisnis, bukan schema response. Contract testing mengisi celah ini dengan verifikasi otomatis sebelum deploy.

E2E tests are too slow and manual review is too error-prone; contract testing provides fast, automated safety gates.

Empat Pilar Consumer-Driven Contract Testing

Consumer menentukan ekspektasi, provider memenuhi, CI memverifikasi, dan gate menolak breaking change.

CONTRACT GENERATOR

1. Consumer Test

Consumer menulis test dengan mock provider, menghasilkan kontrak JSON minimal.

SHARED ARTIFACT

2. Contract File

Kontrak berisi method, path, status, header, body dengan tipe dan casing eksak.

CI GATE

3. Provider Verify

Verifier kirim request sesuai kontrak ke provider; compare status + headers + body subset.

DEPLOYMENT CONTROL

4. Pass/Fail Gate

Hanya pada jalur ini fail → deployment diblokir; pass → deployment diizinkan.

V1+V2 ROUTING

5. Safe Evolution

Dual provider mempertahankan V1 compliant sambil mengekspos V2 schema baru.

Consumer-driven contract testing menggeser inisiatif ke consumer: mereka yang mendefinisikan apa yang dibutuhkan, bukan provider yang mendokumentasikan apa yang tersedia. Hanya pada jalur ini CI gate memblokir deployment ketika verification gagal, mencegah breaking change mencapai produksi.

Consumer drives contract definition; CI gate enforces compliance; dual-version routing enables safe evolution.

Pipeline Kontrak: Dari Consumer Test hingga CI Gate

Alur eksekusi kontrak dari generasi hingga verifikasi di provider side.

STEP 01

{'step': '1', 'label': 'Consumer Test', 'desc': 'Test menjalankan mock server,GenerateMobileContract() membuat kontrak JSON.'}

STEP 02

{'step': '2', 'label': 'Contract.json', 'desc': 'Artefak kontrak tersimpan: MobileApp→OrderService, 1 interaksi, body subset minimal.'}

STEP 03

{'step': '3', 'label': 'CI Trigger', 'desc': 'Provider CI mengambil kontrak, menjalankan verifier.Verify(baseURL, contract).'}

STEP 04

{'step': '4', 'label': 'Verifier Engine', 'desc': 'DiffValues() bandingkan expected vs actual: status, header, body subset, tipe numerik.'}

STEP 05

{'step': '5', 'label': 'Gate Decision', 'desc': 'Passed=true → ALLOWED. Passed=false + errors → BLOCKED with detailed diff report.'}

Pipeline ini dimulai dari consumer test yang menghasilkan artefak kontrak sebagai JSON. Kontrak kemudian diunduh oleh CI provider yang menjalankan verifier engine untuk membandingkan response aktual dengan ekspektasi. Keputusan gate dibuat secara deterministik berdasarkan hasil diff.

Contract generation and verification form a closed loop between consumer and provider in CI/CD.

GenerateMobileContract(): Kontrak Minimal dari Sisi Consumer

Consumer mendeklarasikan hanya field yang benar-benar dibutuhkan.

```
func GenerateMobileContract() *contract.Contract {
    return &contract.Contract{
        Consumer: "MobileApp",
        Provider: "OrderService",
        Interactions: []contract.Interaction{{
            Description:   "A request for order details by ID",
            ProviderState: "Order ORD-123 exists and is IN_PROGRESS",
            Request: contract.RequestDefinition{
                Method: http.MethodGet,
                Path:   "/v1/orders/ORD-123",
            },
            Response: contract.ResponseDefinition{
                Status: http.StatusOK,
                Headers: map[string]string{"Content-Type": "application/json"},
                Body: map[string]interface{}{
                    "id":     "ORD-123",
                    "status": "IN_PROGRESS",
                    "customer": map[string]interface{}{"name": "Budi Santoso"},
                    "total": json.Number("150000"),
                },
            },
        }},
    }
}
```

Fungsi ini menghasilkan kontrak minimal yang hanya mendeklarasikan field: id, status, customer.name, dan total. Field tambahan seperti notes atau created_at tidak dicantumkan karena tidak dipakai consumer — prinsip subset matching memastikan provider dapat berevolusi tanpa merusak kontrak.

Minimal contract: only used fields are declared; extra provider fields are ignored by verifier.

internal/consumer/client.go:84–112

Breaking Provider Menghasilkan 3 Error Detail di CI Gate

Verifikator menangkap enum casing, field rename, dan primitive type mutation.

BREAKING CHANGES TERTANGKAP

Type Mismatch — total - Expected json.Number (integer 150000), got string ('150000'). decoder.UseNumber() mempertahankan presisi tipe numerik.
Value Mismatch — status - Expected 'IN_PROGRESS', got 'in_progress'. Enum casing change terdeteksi sebagai value mismatch.
Missing Field — customer.name - Field 'customer.name' tidak ada di response. Provider mengganti menjadi 'customer.full_name'.

Ketiga error ini dihasilkan oleh diffValues() di verifier.go. Perbedaan tipe numerik tertangkap berkat decoder.UseNumber() yang menjaga json.Number, berbeda dengan default decoder yang mengubah semua angka jadi float64. Field rename terdeteksi karena key tidak ditemukan di expected map.

Json.Number decoder preserves type precision; map key absence detects field renames.

Breaking Change vs Mekanisme Deteksi

Setiap jenis perubahan dipetakan ke mekanisme verifikasi spesifik.

{'mechanism': 'Status Code Check', 'triggers': 'Response bukan 200 OK', 'example': 'Server error atau redirect'}
{'mechanism': 'Header Validation', 'triggers': 'Content-Type != application/json', 'example': 'Provider return text/plain'}
{'mechanism': 'DiffValues Body Subset', 'triggers': 'Missing key / type mismatch / value mismatch', 'example': 'rename, enum change, int→string'}
{'mechanism': 'Json.Number Preservation', 'triggers': 'Integer numeric vs string representation', 'example': "150000 vs '150000'"}

Verifier melakukan tiga lapisan validasi: status code, response header, dan body subset. Lapisan body menggunakan recursive map comparison dengan json.Number untuk membedakan integer dan string numerik. Header validation juga menjadi bagian dari kontrak, bukan hanya payload.

Three-layer validation: HTTP semantics first, then structural diff with type preservation.

Tiga Keputusan Kritis dalam Pipeline Contract Testing

Pass, fail, dan partial-fail masing-masing mengarahkan tindakan berbeda.

DECISION SEQUENCE

Decision 1: Passed?

Hanya pada jalur ini deployment diizinkan. Semua interaction verification harus returning true.

DECISION SEQUENCE

Decision 2: Failed?

Deployment diblokir. Error list harus ditinjau tim provider — bukan sekadar retry tanpa analisis.

DECISION SEQUENCE

Decision 3: Partial?

Beberapa interaction fail, sebagian pass. Review per-interaction untuk изолиasi scope perubahan.

Pipeline contract testing menghasilkan tiga jalur keputusan: pass (deployment izin), fail (deployment blokir dengan error report), dan partial (beberapa kontrak gagal, sebagian lolos). Hanya pada jalur pass saja deployment diizinkan tanpa review tambahan.

Three decision branches: pass allows deploy, fail blocks, partial requires per-interaction review.

Demo 4 Stage: Generate → V1 Pass → Breaking Blocked → Dual Pass

Eksekusi nyata menghasilkan output deterministic yang dapat direproduksi.

```
[Stage 3] Running Breaking Provider Contract Verification:
Result: BLOCKED! Breaking changes detected before deployment:
  1. [A request for order details by ID] path 'total':
     type mismatch (expected 150000 [json.Number], got 150000 [string])
  2. [A request for order details by ID] path 'status':
     value mismatch (expected "IN_PROGRESS", got "in_progress")
  3. [A request for order details by ID] missing expected field
     'customer.name'
CI Deployment Gate: PREVENTED PRODUCTION OUTAGE.
```

Output demo ini menunjukkan Stage 3 yang memverifikasi ProviderBreaking. Tiga error detail tercetak secara terstruktur, masing-masing menyebutkan path field, jenis mismatch, dan nilai yang diharapkan versus aktual. Gate CI menolak deployment — produksi terlindungi.

Deterministic error reporting provides actionable feedback to the provider team.

YANG TIDAK DIBUKTIKAN dalam Laboratorium Ini

Lab adalah simplifikasi; beberapa aspek produksi belum tercover.

BATASAN LAB

Pact Broker Remote - Kontrak dipertukarkan in-memory; tidak ada broker sentral untuk store dan can-i-deploy check lintas tim.
Provider State Management - State didefinisikan via provider_state string; tidak ada callback setup/teardown state fixture.
Array Diff - DiffValues() tidak menangani array comparison — jika response berisi list, diff gagal atau skip.
Message Pact - Verifikasi terbatas pada HTTP REST; event-driven (Kafka/RabbitMQ) tidak diuji dalam lab.
V2 Schema Assertion - Endpoint /v2/orders/{id} hanya diassert status 200, bukan body schema sesuai kontrak.

Laboratorium ini sengaja disederhanakan untuk reproduktibilitas lokal. Beberapa fitur produksi seperti Pact Broker, provider state callbacks, dan array diff tidak diimplementasikan. V2 endpoint hanya diverifikasi status code-nya, bukan body schema — ini adalah gap yang perlu ditambal untuk production readiness.

Lab scope is a minimal viable engine; production requires broker, state management, and broader diff coverage.

research-audit/03-claim-audit.md:1-164

Trade-off: Custom Verifier vs Full Pact Stack

Keputusan pure Go stdlib mengorbankan fitur produksi demi reproduktibilitas.

CUSTOM VS FULL PACT

Custom Verifier (Lab)

Pure Go stdlib, zero external deps, race-safe, cepat. Kompromi: subset matching, tanpa broker, tanpa message pact.

CUSTOM VS FULL PACT

Full Pact Stack (Produksi)

Ruby/JS client, Pact Broker, provider state callbacks, message pact. Lebih lengkap tapi butuh dependensi eksternal.

CUSTOM VS FULL PACT

Hybrid Approach

Gunakan custom verifier untuk CI gate cepat, migrasi ke Pact Broker saat skalabilitas lintas tim diperlukan.

Laboratorium memilih custom verifier dengan Go standard library murni untuk menghindari dependensi Ruby atau Node.js. Kelebihannya: test berjalan di environment apa pun. Kekurangannya: tidak ada broker sentral, provider state management manual, dan tidak support message pact. Untuk produksi skala kecil, custom verifier cukup.

Zero-dependency trade-off sacrifices broker and state management for portability and speed.

Verifier Thread-Safe: 20 Goroutine Tanpa Data Race

Setiap goroutine menginisialisasi request dan response lokal — state tidak dibagikan.

CONCURRENT VERIFICATION PATTERN

Per-goroutine Client - Verifier.NewVerifier() membuat instance baru dengan http.Client timeout 5s — tidak ada shared mutable state.
Request Isolation - http.NewRequest() menghasilkan request object unik per interaksi — concurrent Do() aman.
Result Aggregation - Setiap goroutine mengembalikan VerificationResult lokal — sinkronisasi hanya di wg.Wait(), bukan di result.
Race Detector PASS - go test -race ./... menghasilkan: ok labs/26-contract-testing/tests 1.425s tanpa race detected.

Verifier tidak menggunakan shared mutable state manapun. Setiap panggilan Verify() menginisialisasi http.Client baru, membuat request baru, dan mengembalikan result baru. Race detector Go konfirmasi thread-safety melalui test concurrent 20 goroutine selama 0.01 detik.

Stateless verification per call guarantees thread-safety without locks or mutexes.

tests/contract_test.go:97-116

Sebelum vs Sesudah Contract Testing di Pipeline CI

Transformasi dari blind deploy ke gated deployment dengan evidence.

BEFORE VS AFTER CONTRACT TESTING

Deploy tanpa gate verifikasi kontrak - CI gate memverifikasi kontrak sebelum deploy
Production outage karena field rename - Breaking change terdeteksi di PR, error report jelas
Debug integration failure manual耗时 - Verifikasi otomatis menghasilkan diff terstruktur per field
E2E test berjalan lambat dan flaky - Contract test cepat (sub-second) dan deterministic
Provider evolusi tanpa koordinasi consumer - Dual provider V1+V2 memungkinkan migrasi independen

Sebelum contract testing, tim mengandalkan E2E test yang lambat dan manual debugging saat integrasi pecah. Setelah implementasi, setiap PR provider divalidasi terhadap kontrak consumer — breaking change tertangkap sebelum deploy. Tidak otomatis berarti semua edge case tertutup, tetapi risiko production outage berkurang signifikan.

Contract testing shifts integration safety left from production to CI.

Verifier Engine: Recursive Map Diff dengan Type Preservation

Mekanisme diffValues() menjadi inti deteksi breaking change.

DIFFVALUES ENGINE

decoder.UseNumber() - Mengubah semua angka JSON jadi json.Number, bukan float64. Integer 150000 tetap json.Number, string '150000' tetap string.
map[string]interface{} - Body di-parse sebagai generic map — memungkinkan subset matching tanpa struct tag.
reflect.TypeOf comparison - Primitive type mismatch (string vs int) dideteksi dengan reflect.TypeOf, bukan string comparison.
Recursive traversal - Nested object comparison dilakukan secara rekursif — customer.name vs customer.full_name terdeteksi di level child.
Nil expected handling - Jika expected nil, diffValues() return empty — field opsional diabaikan oleh verifier.

Verifier engine menggunakan pendekatan subset matching: hanya field yang dideklarasikan consumer yang diperiksa. Nested map traversal memungkinkan deteksi error di field dalam (customer.name). UseNumber() adalah kunci yang membedakan integer dan string numerik — fitur yang tidak tersedia pada JSON parser default Go.

Subset matching with recursive diff and json.Number preserves type precision for breaking change detection.

Tujuh Test: Semua PASS, Race Detector Bersih

Verifikasi menyeluruh mencakup generasi, sukses, gagal, dual, concurrent, header, dan v2.

```
=== RUN   TestConsumerContractGeneration
--- PASS: TestConsumerContractGeneration (0.00s)
=== RUN   TestProviderV1_ContractVerification_Success
--- PASS: TestProviderV1_ContractVerification_Success (0.00s)
=== RUN   TestProviderBreaking_ContractVerification_Fails
--- PASS: TestProviderBreaking_ContractVerification_Fails (0.00s)
=== RUN   TestProviderDual_ContractVerification_Success
--- PASS: TestProviderDual_ContractVerification_Success (0.00s)
=== RUN   TestConcurrentContractVerification
--- PASS: TestConcurrentContractVerification (0.01s)
=== RUN   TestVerifier_HeaderValidation_And_ErrorBranches
--- PASS: TestVerifier_HeaderValidation_And_ErrorBranches (0.00s)
=== RUN   TestProviderDual_V2Endpoint_DirectAssertion
--- PASS: TestProviderDual_V2Endpoint_DirectAssertion (0.00s)
ok      labs/26-contract-testing/tests  0.390s
```

Ketujuh test menutupi seluruh skenario: generasi kontrak, verifikasi V1 sukses, breaking change gagal, dual provider sukses, concurrent safety, header validation, dan v2 assertion. Race detector berjalan bersih tanpa flagged race condition.

Comprehensive test coverage validates all verification paths and concurrency safety.

Race Detector: 1.425s untuk Seluruh Test Suite

Konfirmasi thread-safety tanpa mutex atau lock.

```
=== RUN   TestConsumerContractGeneration
--- PASS: TestConsumerContractGeneration (0.00s)
=== RUN   TestProviderV1_ContractVerification_Success
--- PASS: TestProviderV1_ContractVerification_Success (0.00s)
=== RUN   TestProviderBreaking_ContractVerification_Fails
--- PASS: TestProviderBreaking_ContractVerification_Fails (0.00s)
=== RUN   TestProviderDual_ContractVerification_Success
--- PASS: TestProviderDual_ContractVerification_Success (0.00s)
=== RUN   TestConcurrentContractVerification
--- PASS: TestConcurrentContractVerification (0.01s)
=== RUN   TestVerifier_HeaderValidation_And_ErrorBranches
--- PASS: TestVerifier_HeaderValidation_And_ErrorBranches (0.00s)
=== RUN   TestProviderDual_V2Endpoint_DirectAssertion
--- PASS: TestProviderDual_V2Endpoint_DirectAssertion (0.00s)
ok      labs/26-contract-testing/tests  (cached)
ok      labs/26-contract-testing/tests  1.425s
    (race detector: no races detected)
```

Race detector mengonfirmasi bahwa tidak ada shared mutable state antara goroutine. Setiap verifikasi berjalan independent dengan http.Client lokal dan request object unik. Performa concurrent test hanya 0.01s — verifier engine efisien dan scalable.

Thread-safety achieved through stateless per-call design, not synchronization primitives.

Tiga Pola Provider: Compliant, Breaking, dan Dual Evolution

Perbandingan implementasi HTTP handler.

{'pattern': 'ProviderV1', 'route': '/v1/orders/{id}', 'behavior': 'Mengembalikan OrderResponseV1 compliant kontrak. Field notes ada tapi tidak divalidasi (subset matching).'}
{'pattern': 'ProviderBreaking', 'route': '/v1/orders/{id}', 'behavior': 'Mengembalikan OrderResponseBreaking dengan 3 mutasi: enum casing, field rename, type mutation.'}
{'pattern': 'ProviderDual', 'route': '/v1/orders/{id}', 'behavior': 'Mempertahankan V1 route compliant sambil menambahkan V2 route dengan schema baru + currency field.'}
{'pattern': 'ProviderDual', 'route': '/v2/orders/{id}', 'behavior': 'Respons V2: OrderResponseV2 dengan customer.full_name, total string, dan currency IDR. Belum ada kontrak consumer.'}

Tiga pola provider ini menggambarkan spektrum kompatibilitas. ProviderV1 adalah baseline compliant. ProviderBreaking menunjukkan skenario worst-case. ProviderDual merepresentasikan praktik evolusi aman: V1 dipertahankan untuk consumer lama, V2 diperkenalkan untuk consumer baru tanpa mengganggu yang sudah ada.

Dual routing enables independent consumer migration without breaking existing integrations.

Struktur DTO: V1, Breaking, V2 — Satu Domain, Tiga Wujud

Refleksi Go struct dengan json tag menentukan perilaku serialisasi.

DTO STRUCT ANALYSIS

OrderResponseV1 - id, status, customer(Name), total(int64), notes,omitempty - Memenuhi kontrak consumer — semua field expected ada dengan tipe benar.
OrderResponseBreaking - id, status(in_progress), customer(FullName), total(string), notes - Melanggar kontrak — 3 breaking change: enum casing, field rename, type mutation.
OrderResponseV2 - id, status(in_progress), customer(FullName), total(string), currency - Schema baru V2 — belum ada consumer contract, hanya status assertion di test.

Setiap struct merepresentasikan fase evolusi API. V1 adalah kontrak asli yang dipatuhi. Breaking adalah contoh deformasi kontrak yang harus ditangkap. V2 adalah arah evolusi yang sah — field rename dan type mutation diterima karena consumer V2 yang baru telah beradaptasi.

DTO struct tags define serialization behavior; contract testing validates the serialized output against expectations.

Checklist Implementasi Contract Testing yang Benar

Panduan praktis untuk adopsi CDC di tim engineering.

CDC IMPLEMENTATION CHECKLIST

Poin 1

Consumer hanya mendeklarasikan field yang benar-benar digunakan — jangan over-specify.

CDC IMPLEMENTATION CHECKLIST

Poin 2

Kontrak mencakup method, path, status, header, body dengan tipe dan casing eksak.

CDC IMPLEMENTATION CHECKLIST

Poin 3

Provider CI harus menjalankan verifier sebelum merge — pass = boleh deploy, fail = blokir.

CDC IMPLEMENTATION CHECKLIST

Poin 4

Breaking change: tambah field baru dulu, migrasi consumer, baru hapus field lama.

CDC IMPLEMENTATION CHECKLIST

Poin 5

V1 tetap compliant sambil V2 diperkenalkan untuk consumer yang bermigrasi.

CDC IMPLEMENTATION CHECKLIST

Poin 6

Jangan uji aturan validasi bisnis (max 20 chars) di kontrak — fokus pada format pesan.

Checklist ini merangkum praktik terbaik contract testing. Poin terpenting: kontrak harus minimal dan berfokus pada field yang digunakan consumer. Over-specification adalah anti-pattern utama yang membuat kontrak terlalu rapuh dan menghambat evolusi provider.

Minimal contract, CI gate enforcement, and expand/contract pattern are the three non-negotiables.

Lima Anti-Pattern Contract Testing dan Cara Reproduksi

Setiap anti-pattern dapat direproduksi dengan modifikasi kontrak atau provider.

ANTI-PATTERNS & REPRODUCTION

Over-specification - Tambahkan field validasi bisnis (e.g., 'username max 20 chars') ke kontrak. Provider yang melonggarkan validasi akan pecah kontrak tanpa dampak consumer.
Tidak bedakan additive vs breaking - Ubah total dari int jadi string tanpa update kontrak. Verifier harus gagal — jika lolos, matcher terlalu longgar.
Kontrak terlalu ketat - Deklarasikan semua field response (termasuk notes, created_at) sebagai required. Provider tambah field baru akan fail.
JSON Schema saja - Ganti diffValues dengan validator JSON Schema. Casing enum IN_PROGRESS vs in_progress akan lolos Schema valid.
Contract testing menggantikan unit test - Hapus semua unit test internal, hanya jalankan contract test. Logika bisnis tidak terverifikasi.

Kelima anti-pattern ini umum terjadi saat tim baru mengadopsi contract testing. Over-specification adalah yang paling berbahaya karena membuat kontrak terlalu kaku. JSON Schema alone tidak cukup karena tidak menangkap semantik tipe seperti integer vs string numerik.

Anti-patterns arise from misunderstanding contract scope; specificity must match actual consumer usage.

Reproduksi Mandiri: Validasi Full Pipeline di Lokal

Semua tahap demo dapat dijalankan dengan command sederhana tanpa dependensi eksternal.

```
# 1. Jalankan seluruh test suite dengan verbose
go test -v ./...
#
# 2. Jalankan race detector untuk verifikasi thread-safety
go test -race ./...
#
# 3. Eksekusi demo 4 stage (generate -> V1 pass -> breaking blocked -> dual pass)
go run ./cmd/demo
#
# 4. Build binary untuk environment produksi
go build -o bin/contract-demo ./cmd/demo
```

Semua command di atas berjalan di Go standard library tanpa dependensi eksternal. Race detector mengonfirmasi thread-safety concurrent verification. Demo 4 stage memberikan bukti visual yang dapat direproduksi di mesin manapun.

Zero-dependency local verification guarantees fast, repeatable validation across developer workflows.

README.md:30-48

Prinsip Inti: Kendalikan Kontrak, Evolusi dengan Aman

Arsitektur bertahan lama ketika kontrak divalidasi di setiap lapisan deploy.

TIGA PRINSIP ARSITEKTUR JANGKA PANJANG

Consumer Drives the Contract

Consumer yang menentukan ekspektasi minimal — bukan provider yang mendokumentasikan capability. Ini memastikan kontrak relevan dengan kebutuhan riil.

TIGA PRINSIP ARSITEKTUR JANGKA PANJANG

Gate Before Deploy

Verifikasi kontrak harus berjalan di CI sebelum merge. Tidak otomatis berarti semua kasus tercover, tetapi mengurangi risiko production outage secara signifikan.

TIGA PRINSIP ARSITEKTUR JANGKA PANJANG

Dual Routing for Migration

Pertahankan V1 compliant sambil memperkenalkan V2. Consumer bermigrasi sesuai ritme masing-masing tanpa tekanan breaking change.

Prinsip pertama memastikan kontrak selalu berpusat pada consumer — bukan dokumentasi provider yang sering tertinggal. Prinsip kedua menempatkan gate di titik paling kritis: sebelum kode mencapai production. Prinsip ketiga menyediakan mekanisme evolusi tanpa disrupti.

Consumer-driven contracts, CI gates, and dual routing form the triad of sustainable API evolution.

Sistem tanpa contract testing adalah sistem yang menunda kegagalan integrasi produksinya sendiri.

A system without contract testing is merely delaying its inevitable integration failure.

Software Engineering Lab 26 - Contract Testing

internal/consumer/client.go — MobileOrderClient & GenerateMobileContract()
internal/contract/verifier.go — Verifier.Verify() & diffValues()
internal/model/order.go — OrderResponseV1/Breaking/V2 DTOs
internal/provider/server.go — ProviderV1/Breaking/Dual handlers
cmd/demo/main.go — 4-stage demo orchestrator
tests/contract_test.go — 7 test cases

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/26-contract-testing`

Lukman / lukman-ss

Commit: 4a5cf38 • Status: APPROVED
Test passing: 7 unit tests (generation, V1 pass, breaking fail, dual pass, concurrent, header validation, V2 assertion)
Race detector: Clean (go test -race ./... PASS in 1.425s)
Demo: 4-stage executeable (generate → V1 pass → breaking blocked → dual pass)